

DT081A - Signal and Image Processing - Lab 1

Version 1.0 (2024-09-18)

Introduction

This lab provides a hands-on introduction to fundamental concepts in signal and image processing. Through these exercises, you'll gain practical experience with signal generation, Fourier analysis, image manipulation, and audio processing.

Prerequisites

Before starting the lab, ensure you have the following Python libraries installed:

```
pip install numpy matplotlib pillow requests scipy sounddevice
```

1. Signal Generation and Analysis

1.1 Signal Generation and Visualization

Generate and visualize simple 1D signals to understand their properties in the time domain.

```
import numpy as np
import matplotlib.pyplot as plt

# Time vector
t = np.linspace(0, 1, 1000)

# Generate signals
sine_wave = np.sin(2 * np.pi * 10 * t)
square_wave = np.sign(np.sin(2 * np.pi * 5 * t))

# Plot signals
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(t, sine_wave)
plt.title('Sine Wave')
plt.subplot(2, 1, 2)
plt.plot(t, square_wave)
plt.title('Square Wave')
plt.tight_layout()
plt.show()
```

Questions:

1. Modify the code to create a sawtooth wave. Describe the mathematical function you used.
2. Change the frequency of the sine wave to 20 Hz. How does this affect the plot?
3. What happens to the square wave if you change the sign function to `np.ceil(np.sin(2 * np.pi * 5 * t)) * 2 - 1`? Explain why.
4. Create a composite signal by adding the sine wave and square wave together. Plot this new signal and describe its characteristics.
5. How might understanding different signal types be useful in real-world applications? Provide an example.

1.2 Fourier Transform and Frequency Domain Analysis

Implement and visualize the Discrete Fourier Transform to understand frequency domain representation of signals.

```
import numpy as np
import matplotlib.pyplot as plt

def generate_signal(t):
    return np.sin(2 * np.pi * 10 * t) + 0.5 * np.sin(2 * np.pi * 20 * t)
```

```

t = np.linspace(0, 1, 1000)
signal = generate_signal(t)

# Compute DFT
fft_result = np.fft.fft(signal)
freqs = np.fft.fftfreq(len(t), t[1] - t[0])

# Plot
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(t, signal)
plt.title('Time Domain')
plt.subplot(2, 1, 2)
plt.plot(freqs[:len(freqs)//2], np.abs(fft_result)[:len(freqs)//2])
plt.title('Frequency Domain')
plt.xlabel('Frequency (Hz)')
plt.tight_layout()
plt.show()

```

Questions:

1. What frequencies are present in the signal? How can you identify them from the frequency domain plot?
2. Modify the `generate_signal` function to include a 30 Hz component. How does this change the frequency domain representation?
3. What happens to the frequency domain plot if you add random noise to the signal? Implement this and explain your observations.
4. Create a function that generates a signal with a frequency that changes over time (e.g., a chirp signal). Plot this signal in both time and frequency domains and explain what you observe.
5. Describe a practical application where understanding the frequency domain of a signal is important.

1.3 Sampling and Aliasing

Demonstrate the effects of sampling and aliasing on signals.

```

import numpy as np
import matplotlib.pyplot as plt

def generate_and_sample_signal(f, fs):
    t = np.linspace(0, 1, 1000)
    original = np.sin(2 * np.pi * f * t)
    t_sampled = np.arange(0, 1, 1/fs)
    sampled = np.sin(2 * np.pi * f * t_sampled)
    return t, original, t_sampled, sampled

f = 10 # Signal frequency
fs = 15 # Sampling frequency

t, original, t_sampled, sampled = generate_and_sample_signal(f, fs)

plt.figure(figsize=(12, 6))
plt.plot(t, original, label='Original')
plt.stem(t_sampled, sampled, 'r', label='Sampled')
plt.title(f'Signal ({f} Hz) sampled at {fs} Hz')
plt.legend()
plt.show()

```

Questions:

1. What is the Nyquist rate for the given signal? Is the current sampling rate sufficient?
2. Increase the sampling rate to 25 Hz. How does this affect the sampled signal's representation of the original signal?
3. Set the signal frequency to 20 Hz and the sampling rate to 30 Hz. Describe and explain the resulting aliasing effect.
4. Create a function that demonstrates the "wagon-wheel effect" by sampling a high-frequency sinusoid at a much lower rate. Plot the original and sampled signals, and explain the visual effect.
5. Describe a real-world scenario where understanding aliasing is crucial.

2. Image Processing

2.1 Image Sampling and Quantization

Download an image from the provided URL and reduce its detail representation through resampling and requantization.

```

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import requests
from io import BytesIO

# Image URL
image_url = "https://upload.wikimedia.org/wikipedia/en/7/7d/Lenna_%28test_image%29.png"

def load_image(url):
    response = requests.get(url)
    return np.array(Image.open(BytesIO(response.content)).convert('L'))

def resample_image(image, factor):
    small = image[::factor, ::factor]
    return np.repeat(np.repeat(small, factor, axis=0), factor, axis=1)

def requantize_image(image, bpp):
    levels = 2**bpp
    return np.floor(image / (256/levels)) * (256/levels)

# Load image
img = load_image(image_url)

# Process image
factor = 4
bpp = 4

img_resampled = resample_image(img, factor)
img_requantized = requantize_image(img, bpp)

plt.figure(figsize=(15, 5))
plt.subplot(131), plt.imshow(img, cmap='gray'), plt.title('Original')
plt.subplot(132), plt.imshow(img_resampled, cmap='gray'), plt.title(f'Resampled (factor {factor})')
plt.subplot(133), plt.imshow(img_requantized, cmap='gray'), plt.title(f'Requantized ({bpp} bpp)')
plt.savefig('processed_image.png')
plt.show()

```

Questions:

1. How does the resampling factor affect the image quality? At what factor does the distortion become apparent?
2. What is the characteristic of the distortion introduced by resampling? How does it differ from the distortion introduced by requantization?
3. How few bits per pixel can be used in requantization before the image distortion becomes apparent?
4. Implement a function that applies both resampling and requantization to the image. Experiment with different combinations of factors and bits per pixel. What combination gives the best balance between compression and visual quality?
5. Modify the code to implement a more advanced resampling method, such as bicubic interpolation. Compare the results with the simple resampling method used initially. Discuss the trade-offs between image quality and computational complexity.
6. Try using different images by changing the `image_url`. You can find a variety of test images at the USC-SIPI Image Database (<http://sipi.usc.edu/database/>). How do the effects of resampling and requantization vary for different types of images (e.g., portraits, textures, aerial photos)?

2.2 PSNR Calculation

Implement the Peak Signal-to-Noise Ratio (PSNR) calculation and add it to the previous code to allow for objectively measure image quality.

```

import numpy as np

def calculate_psnr(original, compressed):
    mse = np.mean((original - compressed) ** 2)
    if mse == 0:
        return float('inf')
    max_pixel = 255.0
    psnr = 20 * np.log10(max_pixel / np.sqrt(mse))
    return psnr

# Calculate PSNR for resampled and requantized images
psnr_resampled = calculate_psnr(img, img_resampled)
psnr_requantized = calculate_psnr(img, img_requantized)

print(f"PSNR for resampled image: {psnr_resampled:.2f} dB")
print(f"PSNR for requantized image: {psnr_requantized:.2f} dB")

```

Questions:

1. Which process (resampling or requantization) introduces more distortion according to the PSNR values?
2. How well do the PSNR values correlate with your visual perception of image quality?

3. Create a function that calculates PSNR for different resampling factors and bits per pixel. Plot these results and discuss the trade-offs between compression and quality.
4. Research the Structural Similarity Index (SSIM) as an alternative to PSNR. How does SSIM address some of the limitations of PSNR?
5. Implement the SSIM metric and compare its results with PSNR for your processed images. Discuss the differences in how these metrics evaluate image quality.

2.3 Noise Addition and PSNR Analysis

Add normally distributed noise to the image and analyze its effect using PSNR. Again, supplement your previous code to include this functionality

```
import numpy as np
import matplotlib.pyplot as plt

def add_noise(image, std_dev):
    noise = np.random.normal(0, std_dev, image.shape)
    noisy_image = image + noise
    return np.clip(noisy_image, 0, 255).astype(np.uint8)

std_devs = np.linspace(0, 50, 20)
psnr_values = []

for std_dev in std_devs:
    noisy_img = add_noise(img, std_dev)
    psnr = calculate_psnr(img, noisy_img)
    psnr_values.append(psnr)

plt.plot(std_devs, psnr_values)
plt.xlabel('Standard Deviation of Added Noise')
plt.ylabel('PSNR (dB)')
plt.title('PSNR vs. Noise Level')
plt.show()

# Display noisy image at a specific noise level
std_dev_display = 25
noisy_img_display = add_noise(img, std_dev_display)
plt.figure(figsize=(10, 5))
plt.subplot(121), plt.imshow(img, cmap='gray'), plt.title('Original')
plt.subplot(122), plt.imshow(noisy_img_display, cmap='gray'), plt.title(f'Noisy (std dev = {std_dev_display})')
plt.show()
```

Questions:

1. At what PSNR value does the noise start to become visibly apparent in the image?
2. How does this PSNR threshold compare to the often-cited 30-50 dB range for acceptable image quality?
3. At what noise level (standard deviation) does the image become unrecognizable? What is the corresponding PSNR?
4. Implement a simple noise reduction technique, such as a median filter or Gaussian blur. Apply this to the noisy images and compare the PSNR values before and after noise reduction. Discuss the effectiveness and limitations of this simple denoising approach.
5. Research and briefly describe an advanced noise reduction technique. How does it improve upon simpler methods?

3. Audio Processing

3.1 Audio Signal Analysis

Load, visualize, and analyze an audio signal in both time and frequency domains.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
import requests
from io import BytesIO
import sounddevice as sd
from matplotlib.widgets import Button

# Audio URL
audio_url = "https://github.com/parisjava/wav-piano-sound/raw/master/wav/a1.wav"

def load_and_analyze_audio(url):
    response = requests.get(url)
    samplerate, data = wavfile.read(BytesIO(response.content))

    # Ensure data is mono
    if len(data.shape) > 1:
        data = data[:, 0]
```

```

# Normalize data
data = data / np.max(np.abs(data))

# Time array
time = np.arange(0, len(data)) / samplerate

# Compute Fourier Transform
n = len(data)
fft_result = np.fft.fft(data)
freqs = np.fft.fftfreq(n, 1/samplerate)

# Plot waveform, frequency spectrum, and spectrogram
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(12, 15))

# Time domain plot
ax1.plot(time, data)
ax1.set_title('Audio Waveform')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('Amplitude')

# Frequency domain plot
ax2.plot(freqs[:n//2], np.abs(fft_result[:n//2]))
ax2.set_title('Frequency Spectrum')
ax2.set_xlabel('Frequency (Hz)')
ax2.set_ylabel('Magnitude')

# Spectrogram
ax3.specgram(data, Fs=samplerate, NFFT=1024, noverlap=512)
ax3.set_title('Spectrogram')
ax3.set_xlabel('Time (s)')
ax3.set_ylabel('Frequency (Hz)')

plt.tight_layout()

# Function to play sound when button is pressed
def play_sound(event):
    sd.play(data, samplerate)

# Add play button
ax_button = plt.axes([0.81, 0.01, 0.1, 0.05])
button = Button(ax_button, 'Play')
button.on_clicked(play_sound)

plt.show()

# Analyze audio
load_and_analyze_audio(audio_url)

```

Questions:

1. What is the duration of the audio clip? What is its highest frequency component?
2. Describe the main features you observe in the spectrogram. What do the bright areas represent?
3. Modify the code to analyze only the first half second of the audio. How does this change the frequency spectrum?
4. Try using different audio files by changing the `audio_url`. You can find a variety of sound effects and audio samples at <https://www.pacdv.com/sounds/>. How do the waveform, frequency spectrum, and spectrogram differ for different types of audio (e.g., music, speech, environmental sounds)?
5. Implement a function to downsample the audio (reduce the sampling rate). How does this affect the audio quality and the frequency spectrum? At what point does aliasing become noticeable?
6. Research and briefly describe a common audio processing technique (e.g., noise reduction, pitch shifting, time stretching). How might this technique be implemented using the concepts you've learned in this lab?

Conclusion

As you complete this lab, consider the following:

1. How do the concepts of sampling and quantization apply across different types of signals (1D signals, images, audio)?
2. What are the similarities and differences in processing techniques for images and audio signals?
3. How might the tools and techniques you've learned in this lab be applied to real-world problems in fields such as communications, medical imaging, or multimedia systems?

Remember to save your code and results, as you may need to refer to them in future labs or assignments. If you have any questions or encounter any issues during the lab, don't hesitate to ask for clarification or assistance.